

```
!pip install -q transformers torch sentencepiece
```

```
# Optional: check GPU availability
import torch
print("PyTorch version:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())

# If you enabled GPU, this will show details; if not, it's fine to proceed on CPU.
!nvidia-smi || true
```

```
PyTorch version: 2.8.0+cu126
CUDA available: False
/bin/bash: line 1: nvidia-smi: command not found
```

```
from transformers import pipeline
import torch
```

```
model_name = "google/pegasus-cnn_dailymail"
```

```
# pipeline uses device index: 0 for first GPU, -1 for CPU
device_id = 0 if torch.cuda.is_available() else -1
```

```
print(f>Loading pipeline for {model_name} on device_id={device_id} ... (first load downloads model)")
summarizer = pipeline("summarization", model=model_name, tokenizer=model_name, device=device_id)
print("Loaded ✅")
```

```
Loading pipeline for google/pegasus-cnn_dailymail on device_id=-1 ... (first load downloads model)
Some weights of PegasusForConditionalGeneration were not initialized from the model checkpoint at google/pegasus-cnn_dailymail
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
Device set to use cpu
Loaded ✅
```

```
article = """
OpenAI has announced the release of a new AI model designed to assist developers with code generation.
The model, which builds on the success of its previous GPT line, is trained on a large dataset of programming languages,
allowing it to generate functions, debug errors, and even suggest improvements to existing code.
Industry experts say this could significantly speed up software development, but also warn about the need
for careful oversight to prevent incorrect or insecure code from being used in production.
"""

summary = summarizer(article, max_length=60, min_length=20, do_sample=False)
print("=== SUMMARY ===")
print(summary[0]["summary_text"])
```

```
=== SUMMARY ===
OpenAI's GPT model is trained on a large dataset of programming languages .<n>It can generate functions, debug errors, and ever
```

```
articles = [
    """OpenAI has announced the release of a new AI model designed to assist developers with code generation.
    The model, which builds on the success of its previous GPT line, is trained on a large dataset of programming languages,
    allowing it to generate functions, debug errors, and even suggest improvements to existing code.
    Industry experts say this could significantly speed up software development, but also warn about the need
    for careful oversight to prevent incorrect or insecure code from being used in production.""",

    """In a historic agreement, world leaders gathered in Geneva today to sign a climate accord aimed at reducing
    carbon emissions by 50 percent over the next two decades. The accord includes commitments from more than 100 countries
    to transition toward renewable energy, cut back on fossil fuels, and invest in sustainable technologies.
    While environmental groups praised the move as a major step forward, critics questioned whether the goals were realistic
    and if enough funding had been pledged to support developing nations in making the transition."""
]
```

```
summaries = summarizer(articles, max_length=60, min_length=20, do_sample=False)
```

```
for i, s in enumerate(summaries, 1):
    clean_summary = s["summary_text"].replace("<n>", " ")
    print(f"--- Article {i} Summary ---")
    print(clean_summary, "\n")
```

```
--- Article 1 Summary ---
OpenAI's GPT model is trained on a large dataset of programming languages . It can generate functions, debug errors, and even s
```

```
--- Article 2 Summary ---
The accord includes commitments from more than 100 countries . Environmental groups praised the move as a major step forward .
```



```
text_to_summarize = article # or your long article
out = summarizer(text_to_summarize, max_length=60, min_length=20, do_sample=False)[0]["summary_text"]
```

```
out = summarizer(text_to_summarize, max_length=50, min_length=20, do_sample=False) summary_text =
```

```
with open("summary.txt", "w", encoding="utf-8") as f:  
    f.write(out)  
print("Saved summary.txt")
```

Saved summary.txt